

Práctica 4

Jorge Izquierdo y Julio Alfonso de la Torre

Universidad de Alicante

Abstract. En esta memoria se documentan los contenidos de la práctica 4 de la asignatura, en la cual debemos realizar una serie de tareas sobre códigos y paralelización.

1 Intro

Los códigos desarrollados en las tareas se encuentran en el siguiente enlace:
<https://github.com/JulioAlfonsoUA/Practica-4-CAR.git>

2 Tarea 0.1

2.1 ¿Para qué sirve la variable chunk?

La variable chunk define el tamaño de los bloques de iteración que se asignan a los hilos a la hora de dividir el trabajo de un bucle for.

2.2 Explica completamente el pragma:

¿Por qué y para qué se usa `shared(a,b,c,chunk)` en este programa? Para establecer dichas variables como compartidas para todos los hilos. Por defecto, al estar fuera de la zona paralela, son variables compartidas, pero se dicen explícitamente para evitar ambigüedades y hacer más claro el código.

¿Por qué la variable `i` está etiquetada como `private` en pragma? Porque cada uno de los hilos tendrá su propia “`i`”, independiente del índice del bucle. Luego, en la paralelización del bucle, sus iteraciones se reparten entre los hilos. Por ende, cada hilo ejecuta una parte de las iteraciones del bucle, evitando condiciones de carrera sobre la variable “`i`”.

2.3 ¿Para qué sirve `schedule`? ¿Qué otras posibilidades hay?

`Schedule` permite asignar bloques a cada hilo en función de un criterio. En este caso, “`dynamic`” significa que la asignación vendrá dada cuando un hilo se quede sin bloque. Es especialmente útil en bucles con iteraciones con costes variados. También existen otros métodos de asignación.

Por un lado está “`static`”, el cual reparte los bloques de manera equitativa entre los hilos antes de iniciar el bucle. Sirve mucho cuando se tratan con bucles donde las iteraciones son uniformes en cuanto al coste.

Por otro lado está “guided”, que asigna los bloques durante la ejecución del bucle de tal manera que primero otorga bloques grandes y va reduciendo su tamaño progresivamente. Es una buena forma de equilibrar carga y también evitar la sobrecarga administrativa de “dynamic”. Se utiliza especialmente en bucles donde algunas iteraciones son más pesadas que el resto.

Por último está “runtime”, el cual determina el tipo de asignación de bloques (“static”, “dynamic” o “guided”) durante la ejecución del programa con la variable `OMP_SCHEDULE`.

2.4 ¿Qué tiempos y otras medidas de rendimiento podemos medir en secciones de código paralelizadas con OpenMP?

OpenMP permite medir tiempos de diferente manera según la clase de información que se necesite.

omp_get_wtime(): Devuelve el tiempo de reloj actual en el momento en el que se invoca la función (expresado en segundos). Para medir el tiempo transcurrido en una sección se llama dos veces (antes y después) y se resta el resultado.

Speedup: Al hablar de paralelismo, siempre hay un speedup que relaciona lo que tarda el programa paralelizado con su versión secuencial. Indica cuantas veces el programa paralelo es más rápido que el secuencial.

omp_get_num_threads(): Retorna el número de hilos activos de la región paralela actual. Es útil para saber el comportamiento del programa y nos permite saber la eficiencia de los hilos gracias al speedup calculado con la ecuación $(S / n_hilos) * 100$.

omp_get_thread_num(): Devuelve el identificador del hilo actual (0 es el hilo maestro). Es útil para analizar el balanceo de carga, ya que nos permite comprobar cada uno de los hilos y ver si están procesando cantidades equitativas de trabajo.

3 Tarea 0.2

3.1 ¿Para qué sirve el parámetro `std::launch::async`?

`std::launch::async` es una política de `std::async` que garantiza la creación de un nuevo hilo de forma inmediata. Permite paralelismo real, ya que ejecuta la tarea en segundo plano sin bloquear el flujo principal del programa. Su uso explícito permite que el sistema evite posponer la tarea, otorgando un control total sobre la asincronía.

3.2 **Calcula el tiempo que tarda el programa con `std::launch::async` y `std::launch::deferred`. ¿A qué se debe la diferencia de tiempos?**

Con `std::launch::async` obtuvimos un tiempo de ejecución de 3.000 ms, mientras que con `std::launch::deferred` obtuvimos 5.000 ms.

Se debe a que con `std::launch::deferred`, la ejecución de task 2 no comenzará hasta que se llame a `task2.get()`, lo que hace que pase de paralelizar task1 y task2 a ejecutarse completamente secuencial.

3.3 **¿Qué diferencia hay entre los métodos `wait` y `get` de `std::future`?**

Ambos esperan para gestionar el resultado de una tarea asíncrona, pero `get()` devuelve el resultado de la función a la que llama y no se puede volver a llamar después. Además, `wait()` se puede ejecutar múltiple veces mientras que `get()` solo una vez.

3.4 **¿Qué ventajas ofrece `std::async` frente a `std::thread`?**

Ambas trabajan con hilos, pero `async` tiene ciertas ventajas sobre `thread`. `Async` permite obtener el resultado de la función mediante un `get()` del objeto `future` que devuelve, mientras que `thread` requiere de comunicación entre hilos mucho más complicada. `Async` devuelve las excepciones como resultado del `get()`, permitiendo tratarlas debidamente, mientras que `thread` hará un `terminate()` en el momento en el que se encuentre un error.

4 **Tarea 0.3**

4.1 **¿Cuál de las dos formas de inicializar el vector y rellenarlo es más eficiente? ¿Por qué?**

La segunda forma de inicializar el vector es mucho más eficiente. El problema que tiene paralelizar la primera versión es que contiene condiciones de carrera por el uso de un vector dinámico con `push_back()`. Dicho vector puede sufrir relocalizaciones al superar cierto tamaño, invalidando los punteros de los otros hilos. En cambio, un vector con tamaño predimensionado evita todos estos problemas.

4.2 **¿Podría ocurrir algún problema al paralelizar los dos bucles `for`? ¿Por qué?**

El problema al paralelizar lo da la primera opción, puesto que como no tiene una sección definida, si un hilo tiene que mover los datos por que se ha quedado sin espacio y otro hilo intenta acceder a esa memoria que ya no es válida o está siendo modificada, el programa dará un error.

5 Paralelización del análisis forense de manipulación de imágenes digitales

5.1 Analiza el código e identifica los distintos procesos

Tras analizar el archivo main secuencial hemos identificado cinco procesos principales que se ejecutan sobre la imagen de entrada:

- `compute_srm(image, 3)`: Análisis de ruido SRM con kernel 3x3.
- `compute_srm(image, 5)`: Análisis de ruido SRM con kernel 5x5.
- `compute_ela(image, 90)`: Error Level Analysis con calidad JPEG 90.
- `compute_dct(image, 8, true)`: DCT inversa con bloques de 8x8.
- `compute_dct(image, 8, false)`: DCT directa con bloques de 8x8.

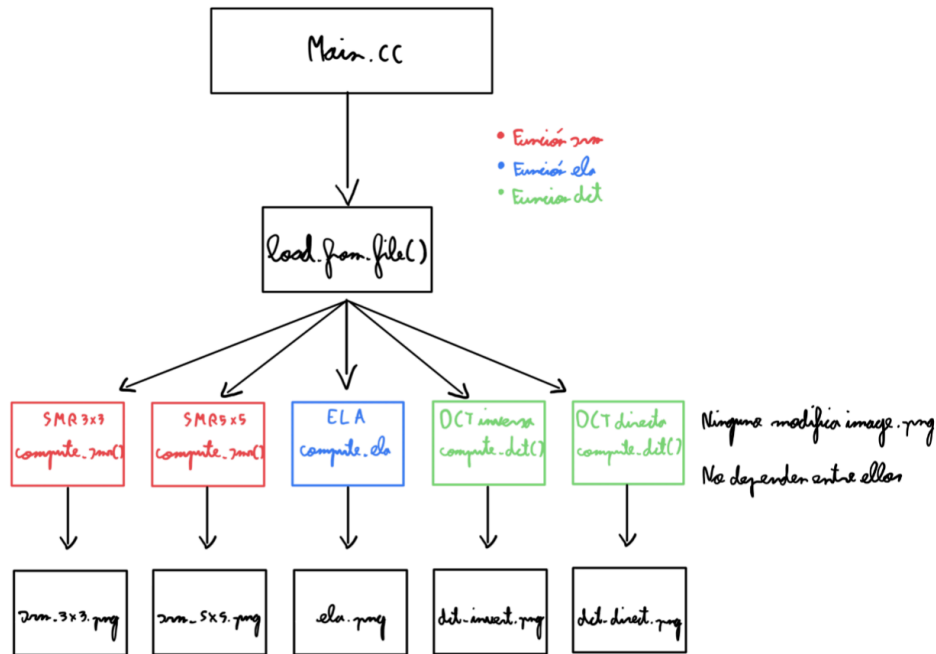


Fig 1: Análisis de dependencias del código

El análisis de dependencias revela que todos los procesos identificados anteriormente comparten lo siguiente:

- Todos leen la imagen de entrada, sin modificarla
- Cada proceso escribe en un fichero de salida diferente (`srm_kernel3x3.png`, `srm_kernel5x5.png`, `ela.png`, `dct_invert.png`, `dct_direct.png`).

- No existe una dependencia de datos entre ellos.

Estas condiciones implican que todos son independientes entre sí y pueden ejecutarse en paralelo sin ningún tipo de problema. Esto se explotará cuando diseñemos la versión paralela con `std::async`.

5.2 Analiza el código de cada proceso y el tiempo

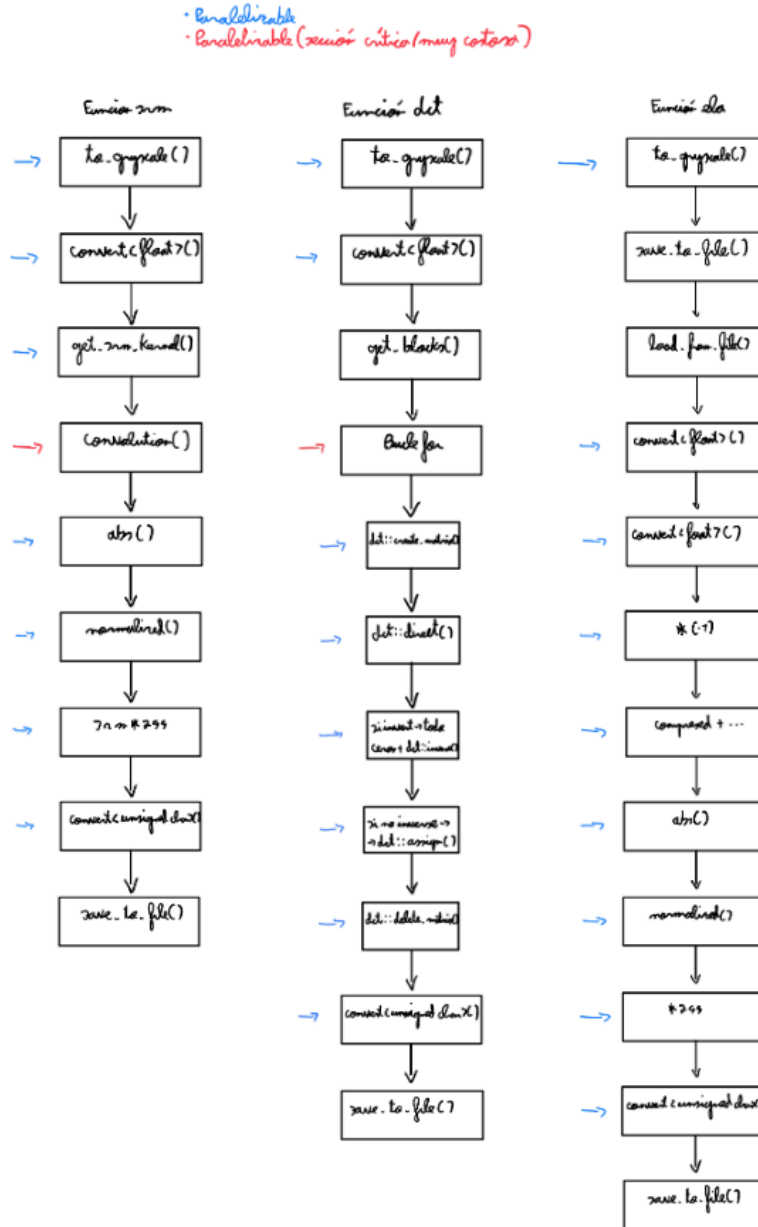


Fig 2: Diagrama de flujo del programa.

En este esquema tenemos cada una de las funciones segmentadas en distintos bloques en orden de llamada, donde los bloques señalados en azul representan funciones u operaciones que se pueden paralelizar y las rojas las secciones paralelizables críticas. Analicemos una a una:

- `compute_srm()`: Cada una de las funciones u operaciones aritméticas de las que está compuesta son perfectamente paralelizables, destacando el bloque convolucional ya que, como vimos en trabajos anteriores, se trata de una función extremadamente costosa.
- `compute_dct()`: Esta función parte de manera similar a `compute_srm()` pero en ella se encuentra un bucle for donde ejecuta una serie de funciones (también compuestos de bucles for) trabajando con bloques creados anteriormente. Debido a esto, dicho bucle for se marcó como muy costoso.
- `compute_ela()`: Similar a `compute_srm()` (llama casi a las mismas funciones), pero es la única que contiene funciones de I/O, por lo que nos será un problema ya que no se puede acelerar dicho proceso.

Calculamos los tiempos de ejecución, medido en microsegundos ($1 \text{ ms} = 1000 \mu\text{s}$), de cada función en microsegundos modificando el archivo main del repositorio. Los resultados son los siguientes:

Cargando imagen...
 [load_from_file] 13546 us
 Dimensiones: 680x680
 canales: 4
 Num bloques DCT (8x8): 7225

Función	tiempo (μs)
SRM 3x3	25391
SRM 5x5	43774
ELA	17444
inverse DCT 8x8	605597
direct DCT 8x8	308720

Fig 3: Tabla de tiempos de ejecución de cada función en microsegundos.

Tiempo total secuencial: 1136556 μs

Estos resultados demuestran la importancia de paralelizar las funciones DCT (especialmente) y SRM ya que consumen gran parte del tiempo de ejecución (debido a que contienen partes muy complejas) frente a la función ELA ya que no contiene operaciones costosas.

5.3 Paralelización

Para determinar qué secciones valen la pena paralelizar hemos modificado el archivo main para que analice el código de cada una de las secciones (o un conjunto de ellas). Para evitar mucho código repetido hemos diseñado unas mini funciones que inicializan el tiempo e imprime el tiempo consumido de cada bloque.

Los resultados son los siguientes:

Cargando imagen...
 [load_from_file] 13546 us
 Dimensiones: 680x680
 canales: 4
 Num bloques DCT (8x8): 7225

función	tiempo (µs)	
	SRM 3X3	SRM 5X5
to_grayscale + convert<float>	2992	1837
convolution	16458	37221
abs	1449	823
normalized	2372	1808
srm * 255	905	860
convert<unsigned char>	1181	1151
Tiempo total(+ overhead)	25391	43774

Fig 4: Tabla de tiempos de ejecución de las funciones internas de SRM

ELA	
función	tiempo (µs)
to_grayscale	1611
save_to_file (I/O escritura)	1658
load_from_file (I/O lectura)	2633
resta (+ convert<float>)	5357
abs	1122
normalized * 255	3834
convert<uchar>]	1146
Tiempo total(+ overhead)	17444

Fig 5: Tabla de tiempos de ejecución de las funciones internas de ELA

función	tiempo (µs)	
	inverse DCT 8x8	direct DCT 8x8
convert<float> + to_grayscale	5253	5014
get_blocks	54	56
bucle DCT	599566	303021
convert<unsigned char>	689	590
Tiempo total(+ overhead)	605597	308720

Fig 6: Tabla de tiempos de ejecución de las funciones internas de DCT

Tiempo total secuencial: 1136556 µs

Podemos observar que las secciones que marcamos en rojo en el apartado 1.2 son justamente las más costosas, por lo que su paralelización es bastante justificable.

El problema llega con el resto de funciones. Casi todas ellas aplican operaciones a la totalidad de la imagen y, siendo que esta tiene un tamaño de 680x680 (unos 400.000 píxeles), consideramos que son lo suficientemente costosas como para intentar paralelizarlas. Sin embargo, este fue el resultado con 8 hilos al paralelizarlo frente a solo paralelizar las secciones críticas:

Paralelización	Tiempo (ms)	Speedup
Casi total	341,57	1
Solo secciones críticas	190,97	1,78860554

Fig 7: Tabla de diferencias en speedup

Como se puede observar, la paralelización de dichas funciones empeora drásticamente la eficiencia de las funciones en general. Parece ser que hemos subestimado el coste de fork-join de los hilos. Concluimos que, pese a que son cálculos un poco grandes para un solo núcleo, no vale la pena paralelizar las funciones que actúan sobre todos los píxeles de la imagen (funciones como `abs()`, `normalize()`, etc. Y las operaciones de suma y multiplicación).

Las únicas funciones que no paralelizaremos seguro son las vinculadas al archivo `dct.cc`, ya que trabajan con bloques de tamaño reducido (8x8) y los bucles son bastante pequeños, por lo que el overhead de paralelización de OpenMP empeoraría claramente su eficiencia. Además, todas estas funciones ya pertenecen al bucle `for` de `compute_dct()` por lo que si la paralelizamos se generaría muchas más sincronizaciones y, por ende, más overhead.

La paralelización completa no da un mejor rendimiento. En muchos casos, el overhead introducido por la gestión de hilos supera el tiempo de ejecución secuencial de la propia tarea. También, si una subtarea es mucho más breve que otras ejecutadas en paralelo, su optimización es irrelevante. Por ende, la estrategia óptima consiste en paralelizar únicamente aquellas secciones críticas donde la reducción del tiempo de ejecución compense el coste de paralelizarlas.

En nuestro caso específico, ELA no se paraleliza dentro de la misma función, pero se ejecuta asincrónicamente junto con las otras dos funciones. El hecho de que no se paralelice ELA es por lo explicado anteriormente sobre que las tareas breves no afectan al tiempo total de ejecución.

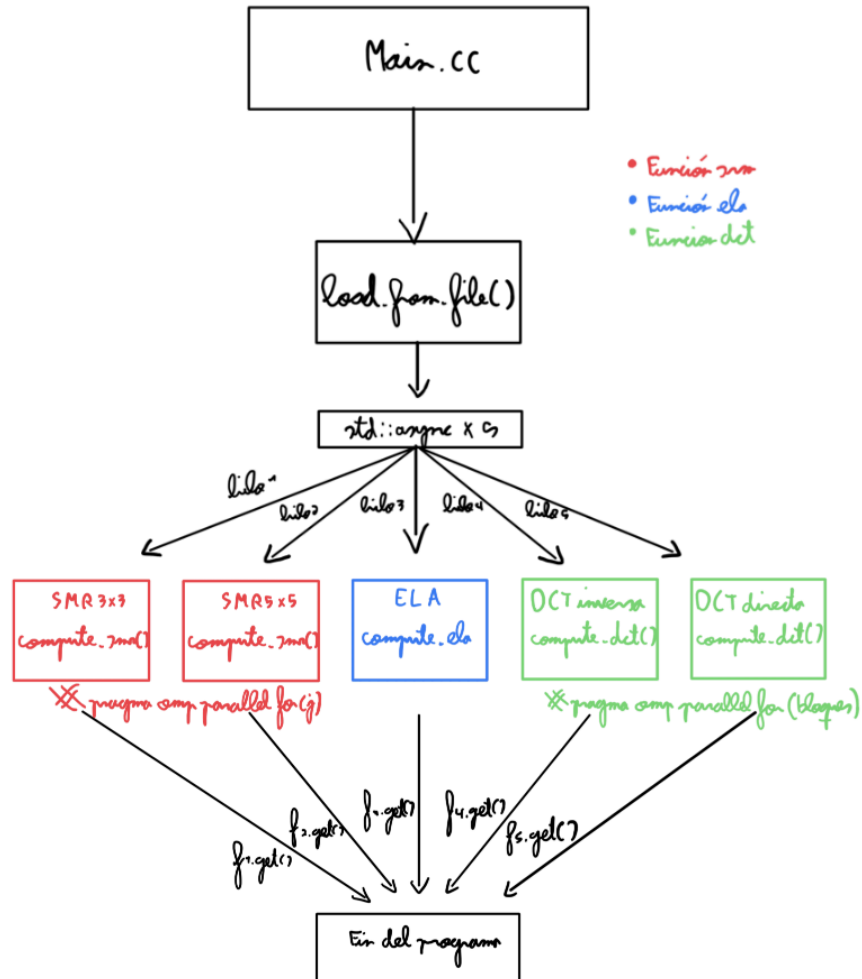


Fig 8: Diagrama de nuestra versión paralelizada

En nuestro caso hemos explotado dos niveles de paralelismo de forma complementaria:

En primer lugar, el paralelismo funcional con `std::async`. Los cinco procesos se lanzan al mismo tiempo como tareas asíncronas mediante `std::async(std::launch::async, ...)`. Dado que todos leen de image en modo lectura y escriben en distintos ficheros, no existe ningún tipo de condición de carrera. La barrera de sincronización se establece con la llamada a `.get()` al final del main().

Este es uno de los mayores impactos: en la versión secuencial el tiempo total viene por la suma de los cinco procesos (1136,56 ms), mientras que en paralelo el tiempo final viene dado por el proceso más largo.

En segundo lugar, el paralelismo de datos con OpenMP. Dentro de cada proceso se aplicaron directivas OpenMP en los bucles de mayor coste:

- `#pragma omp parallel for schedule(static)` en el bucle más externo de la función `convolution()`, en `image.h`. Se escogió “static” porque todas las iteraciones de la convolución tienen un coste bastante uniforme. Las variables declaradas dentro del bucle son automáticamente privadas por ser locales al bucle. La variable “convolved” es shared ya que cada hilo escribe en posiciones diferentes y “kernel” también lo es porque ninguno de los hilos lo modifica, sino que todos ellos necesitan acceder a ella. Por último, usamos `firstprivate` con `kernel_size` para que cada hilo tenga su propia copia inicializada con el valor original, garantizando que las lecturas son locales a cada hilo.
- `#pragma omp parallel for schedule(static)` en el bucle de bloques de la función `compute_dct()`. Usamos “static” porque todas las iteraciones tienen un coste bastante uniforme al usar siempre bloques de 8x8. La variable `dctBlock` es local en cada iteración, por lo que cada hilo tiene su propia copia y así evitamos condiciones de carrera. La variable “blocks” es shared porque cada hilo actúa sobre distintas posiciones del vector. Por último, “block_size” e “invert” son `firstprivate` para que cada hilo tenga su propia copia inicializada con el valor original, garantizando que las lecturas son locales a cada hilo.

A continuación mostramos los resultados de la paralelización definitiva con `std::async` más la paralelización de los dos bucles con OpenMP:

Hilos	Tiempo (ms)	Speedup
1	630,89	1,80151849
2	357,8	3,176523197
4	243,6	4,665681445
8	190,97	5,951510708
16 (máximo)	172,16	6,601765799

Fig 9: Tabla de speedup según la cantidad de hilos

Tiempo secuencial de referencia: 1.136,56 ms.

Con `OMP_NUM_THREADS=1` el speedup es de 1.801, es decir, la versión paralela con un solo hilo es ya mucho más rápida que la versión secuencial pura. Esto se debe a que `std::async` sigue lanzando los cinco procesos en paralelo independientemente del número de hilos. El paralelismo funcional no solo demuestra su gran eficacia, sino que también es muy descriptiva al delatar donde se encuentra el cuello de botella, ya que su tiempo es casi idéntico al tiempo de ejecución de la función más costosa: `compute_dct` en modo `inverse` (630.89 ms frente a 605.60 ms respectivamente).

El speed-up crece notoriamente de uno a cuatro hilos (1.801 a 4.665). Sin embargo, a partir de los ocho hilos, la mejora relativa decrece: de ocho a dieciséis hilos pasa de 190.97 a 172.16 ms (+10% aproximadamente). Esto es coherente con la Ley de Amdahl, el cual estipula que la fracción secuencial no paralelizable (I/O de la función ELA, inicializadores como `get_blocks()`, overhead de fork-join, etc.) limita el speedup máximo alcanzable.

Sobre el código implementado:

La talla del problema viene definida por el tamaño de la imagen de entrada, el tamaño del kernel de la convolución y la cantidad de técnicas utilizadas para la detección de manipulación.

Las estructuras de control utilizadas han sido el mutex y `lock_guard` para proteger las escrituras por consola con `std::cout` al imprimir los tiempos de ejecución de cada función. Como todas ellas se ejecutan en paralelo con `std::async`, sin esta protección las salidas podrían entrelazarse y producir texto incoherente o ilegible.

En el archivo `main`, las cinco llamadas (`compute_srm()`, `compute_ela()` y `compute_dct()`) se sustituyeron por cinco funciones asíncronas (`std::async(std::launch::async, ...)`) que las lanza simultáneamente, añadiendo las llamadas a `.get()` al final para sincronizar.

En `image.h`, se añadió `#pragma omp parallel for schedule(static) shared(convolved, kernel) firstprivate(kernel_size)` en el bucle externo de `convolution()`. Dentro de la función `compute_dct()`, se añadió `#pragma omp parallel for schedule(static) shared(blocks) firstprivate(block_size, invert)` en el bucle de bloques. Están explicadas detalladamente más arriba en la memoria.

El resto del código secuencial se mantuvo intacto, ya que nuestro análisis de tiempos de ejecución mostraron que las demás secciones no mejoraron al ser paralelizados porque el overhead supera la ganancia.

Sobre el paralelismo explotado:

Nuestro código utiliza paralelismo de datos y funcional. Al dividir los bucles `for` de `convolution()` y de `compute_dct()` en distintos hilos estamos haciendo que el mismo conjunto de instrucciones actúe sobre conjunto de datos distintos. El paralelismo funcional se emplea con `std::async` que permite ejecutar funciones completamente independientes de manera paralela, incrementando notoriamente la eficiencia del programa.

El programa combina dos modos de programación paralela. A nivel funcional (`std::async`) se usa MPMD, ya que cada hilo ejecuta una función diferente sobre conjuntos de datos distintos. A nivel de datos con OpenMP, se usa SPMD porque todos los hilos ejecutan el mismo código del bucle pero cada uno actúa sobre un conjunto de datos diferente. La comunicación es implícita mediante variables compartidas en ambos casos, sin ningún paso de mensajes explícito.

La comunicación paralela de este programa depende del momento. Para el inicio de las tareas, cuando se pasa la imagen por referencia, sigue una comunicación uno a todos. En el momento en el que se hace el `get()` de cada proceso donde se estaba calculando cada tarea, la comunicación es todos a uno. Nuestro código hace uso de variables compartidas mediante `shared(blocks)` o `shared(convolved, kernel)`.

Las estructuras paralelas empleadas son master-slave y descomposición de dominio: Master-slave, ya que el hilo master asigna trabajo a los otros hilos (creados con `std::async`), y descomposición del dominio, porque dentro de la función DCT dividimos el trabajo del bucle en secciones de la imagen que son independientes entre sí.

Sobre los resultados de las pruebas realizadas y su contexto:

El ordenador usado tiene las siguientes características:

```

CPU(s): 16
Lista de la(s) CPU(s) en línea: 0-15
Nombre del modelo: 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz
Hilo(s) de procesamiento por núcleo: 2
<<Socket(s)>>: 1
CPU(s) factor de escala MHz: 47%
Indicadores ¿Contiene ht?: Si
Cachés (suma de todas):
    L1d: 384 KiB (8 instancias)
    L1i: 256 KiB (8 instancias)
    L2: 10 MiB (8 instancias)
    L3: 24 MiB (1 instancia)
CPU(s) del nodo NUMA 0: 0-15
Modo(s) NUMA: 1
Aportación del usuario: portátil en modo rendimiento + con cargador puesto
  
```

La salida del comando `lscpu` incluye “Thread(s) per core”, que indica el número de hilos que contiene cada uno de los núcleos físicos. Podemos confirmarlo porque dentro del apartado “Indicadores” contiene el flag “ht”, el cual indica que el procesador aplica Hyper-Threading (HT), que significa que el núcleo tiene capacidad de explotar el paralelismo a nivel de hilos.

Como tiene ocho núcleos físicos y dos hilos por núcleo, la ganancia de paralelizar de ocho a dieciséis hilos es menor frente al resto ya que los hilos adicionales tienen que compartir recursos del mismo núcleo. Por tanto, no llegan a duplicar el rendimiento.

A continuación, mostraremos unas gráficas de speedup ganado gracias al paralelismo tanto en función del número de hilos como las dimensiones de la imagen seleccionada.

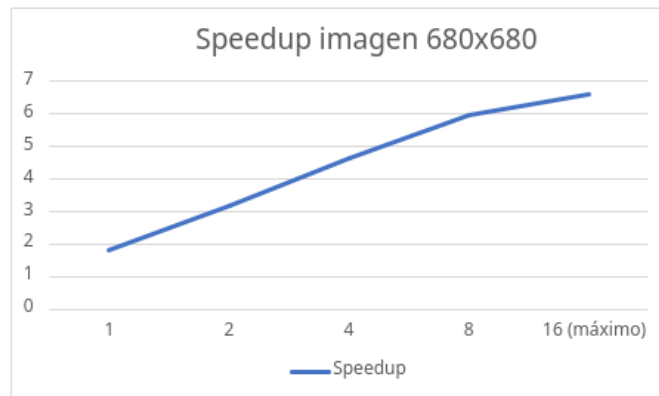


Fig 10: Gráfica de speedup para una entrada de 680x680

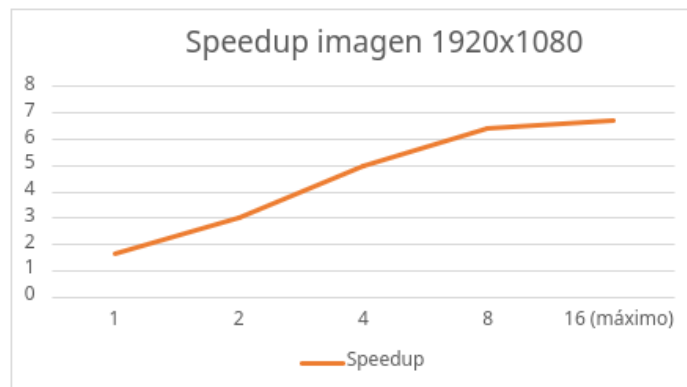


Fig 11: Gráfica de speedup para una entrada de 1920x1080

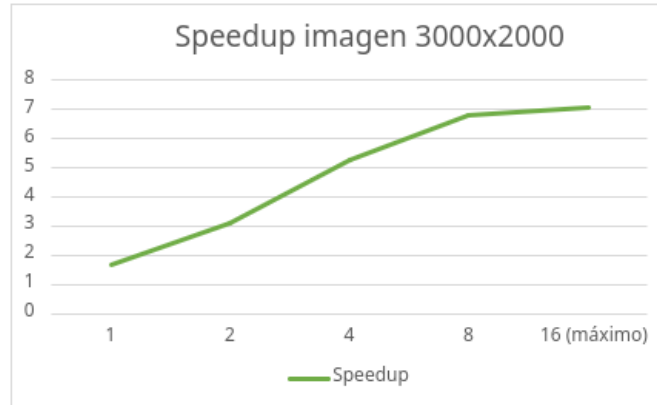


Fig 12: Gráfica de speedup para una entrada de 3000x2000

El primero se ejecutaron las pruebas sobre la imagen que ya venía en el trabajo, de dimensiones 680x680, con un tiempo secuencial de 1.136,56 ms. Se puede observar lo mismo que llevamos explicando durante todo el trabajo: la ganancia significativa de emplear `std::async`, la aceleración fruto de aumentar los hilos y el tope de speedup a partir de los ocho hilos.

El segundo se obtuvo los datos al emplear una imagen de calidad 1080p (1920x1080), con un tiempo secuencial de 4.967,55 ms. Pese a que el tiempo de ejecución aumente, ya que el tamaño de la matriz es mayor, el speedup relativo se mantiene casi idéntico.

El tercero se obtuvo con una imagen de calidad similar a 4k (3000x2000), con un tiempo secuencial de 12751.75 ms. Mismas conclusiones que con la segunda imagen.

La ganancia máxima teórica está dictada por la Ley de Amdahl, la cual establece que la aceleración de un programa está limitada por su fracción secuencial, es decir, aquella parte del código que no puede ejecutarse en paralelo. Por mucho que se aumente el número de procesadores, el tiempo de ejecución nunca podrá ser menor que el tiempo que tarda en completarse esa parte estrictamente lineal. Nuestros cálculos fueron los siguientes:

Cálculo para a_{max}

$$a_g = \frac{T_{secuencial}}{T_{paralela}} = \frac{1736'56ms}{172'16ms} = 6'601$$

$$a_g = \frac{1}{(1-\gamma) + \frac{T}{W}} = \frac{1}{\frac{W(1-\gamma) + T}{W}} = \frac{W}{W + T(1-\gamma)} \rightarrow a_g \cdot W + a_g T(1-\gamma) = W \rightarrow a_g T(1-\gamma) = W - a_g W$$

$$\gamma = \frac{W - a_g \cdot W}{a_g \cdot (1 - W)} = \frac{26 - 6'601 \cdot 76}{6'601 \cdot (1 - 76)} = 0'205 \rightarrow 20'5\%$$

$$a_{max} = \lim_{W \rightarrow \infty} \frac{1}{(1-\gamma) + \frac{T}{W}} = \frac{1}{(1-\gamma) + 0} = \frac{1}{1-\gamma} = \frac{1}{1-0'205} = \frac{1}{0'795} \approx 10'5$$

Fig 13: Cálculo aceleración máxima teórica del código

Hemos utilizado el tiempo de ejecución más pequeño de los paralelizables porque reduce el ruido provocado por factores externos (SO, I/O, etc.).

Como podemos observar, la ganancia máxima a la que podríamos llegar sería de unos 10.5 que está bastante lejos del speedup más alto que hemos obtenido (6.601). Esto significa que estamos bastante lejos del límite teórico, por lo que si ejecutamos este mismo programa en una máquina con más hilos podríamos sacar más partido a la paralelización.

Por otro lado, como comentábamos antes, hemos notado que el speedup relativo de cada una de las imágenes se mantiene casi idéntico. Esto se debe a que las operaciones secuenciales también crecen proporcionalmente al tamaño al igual que las partes paralelizadas. Por esta razón, el speedup se mantiene casi igual.

¿Cuál es la implementación más eficiente de las 2?

La implementación más eficiente es la actual: la combinación de paralelismo funcional con `std::async` junto con paralelismo de datos con OpenMP. Aunque la pregunta pueda resultar ambigua, podemos compararla con otras versiones o implementaciones:

- Respecto a la versión segmentada, es claro que es más eficiente nuestra versión actual, como hemos estado corroborando durante la memoria con gráficos, cálculos, etc. Sin embargo, paralelizar implica utilizar la totalidad de la CPU para una tarea en concreto. Por tanto, la implementación segmentada podría ser útil cuando no tenemos gran interés en la eficiencia del cálculo o debemos ejecutar otra serie de procesos sin monopolizar el procesador. En caso contrario, donde queramos maximizar la eficiencia, la paralelización es una buena idea.
- Respecto a la versión casi totalmente paralelizada, ya demostramos anteriormente que con solo las secciones críticas (`convolution()` y bucle de `compute_dct()`) es más eficiente que intentar paralelizar todo, como demostramos anteriormente en la tabla de la Fig 7. La razón es el overhead de `fork-join`, que en operaciones poco costosas puede empeorar su eficiencia. Además, ELA no se necesitaba paralelizar porque su coste es irrelevante frente a otras funciones como DCT.

Si se compara la contribución de cada técnica empleada en este trabajo, el de mayor impacto es `std::async`, ya que con un hilo ya obtuvimos un speedup de 1,801 simplemente por el paralelismo funcional. OpenMP añade un speedup adicional significativo pero requiere más esfuerzo de implementación y análisis para evitar empeorar el rendimiento del código. Por tanto, si solo se pudiera escoger una de las dos técnicas, `std::async` sería la más eficiente en términos de coste-beneficio a la hora de implementarlo. Sin embargo, es claro que la combinación de ambas es la opción más óptima.

En conclusión, con un buen análisis del código y estudiando las distintas partes paralelizables del código se puede obtener un buen rendimiento del mismo con unas pequeñas modificaciones.